HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 04

Experiment # 04

C++ programming using Structure

Objective:

The objective/objectives of this lab is/**are**

- Declaration of structure.
- Initialization of structure.
- Assigning values to structure data types.
- Self defined variables.
- Reinforcement of the concepts of iterations in structure.

Software Tools:

Code Blocks/ Dev C++/ Visual Studio

Theory:

Introduction:-

C++ provides another structured data type, called a struct (some languages use the term "record"), to group related items of different types. An array is a homogeneous data structure; a struct is typically a heterogeneous data structure.

Structures:-

Suppose that you want to write a program to process student data. A student record consists of, among other things, the student's name, student ID, GPA, courses taken, and course grades. Thus, various components are associated with a student. However, these components are all of different types. For example, the student's name is a string, and the GPA is a floating-point number. Because these components are of different types, you cannot use an array to group all of the items associated with a student. C++ provides a structured data type called **struct** to group items of different types. Grouping components that are related but of different types offers several advantages. For example, a single variable can pass all the components as parameters to a function.

struct: A collection of a fixed number of components in which the components are accessed by

name. The components may be of different types.

The components of a **struct** are called the members of the **struct**. The general syntax of a **struct** in C++ is:

```
struct structName

{

dataType1 identifier1;
dataType2 identifier2;
.
.
.
.
dataTypen identifiern;

};
```

In C++, **struct** is a reserved word. The members of a **struct**, even though they are enclosed in braces (that is, they form a block), are not considered to form a compound statement. Thus, a semicolon (after the right brace) is essential to end the **struct** statement. A semicolon at the end of the **struct** definition is, therefore, a part of the syntax.

The statement:

```
struct houseType

{

string style;

int numOfBedrooms;

 int numOfBathrooms;

  int numOfCarsGarage;

 int yearBuilt;
```

```
int finishedSquareFootage;

double price;

double tax; };
```

defines a **struct houseType** with 8 members. The member **style** is of type **string**, the members **numOfBedrooms**, **numOfBathrooms**, **numOfCarsGarage**, **yearBuilt**, and **finishedSquareFootage** are of type**int**, and the members **price** and **tax** are of type**double**.
Like any type definition, a **struct** is a definition, not a declaration. That is, it defines only a data type; no memory is allocated.

Once a data type is defined, you can declare variables of that type.

For example, the following statement defines **newHouse** to be a struct variable of type **houseType**:

```
//variable declaration

houseType newHouse;
```

The memory allocated is large enough to store **style, numOfBedrooms,numOfBathrooms, numOfCarsGarage, yearBuilt, finishedSquareFootage, price,** and **tax. (See Figure 1)**

Figure 1: struct newHouse

Accessing struct Members:-

In arrays, you access a component by using the array name together with the relative position (index) of the component. The array name and index are separated using square brackets. To access a structure member (component), you use the **struct** variable name together with the member name; these names are separated by a dot (period). The syntax for accessing a **struct** member is:

Struct VariableName.memberName

The **struct VariableName.memberName** is just like any other variable. For example, **newStudent.courseGrade** is a variable of type **char**, **newStudent.firstName** is a string variable, and so on. As a result, you can do just about anything with **struct** members that you normally do with variables. You can, for example, use them in assignment statements or input/output (where permitted) statements.

In C++, the dot (.) is an operator called the member access operator.

Consider the following statements:

```
struct studentType

{

string firstName;

string lastName;

char courseGrade;

int testScore;

int programmingScore;

double GPA;

};
```

HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 04

```
//variables in main()

studentType  newStudent;

studentType  student;
```

Suppose you want to initialize the member **GPA** of **newStudent** to **0.0**. The following statement accomplishes this task:

```
newStudent.GPA = 0.0;
```

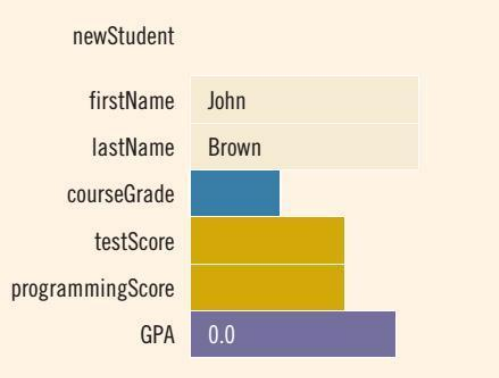
Similarly, the statements:

```
newStudent.firstName = "John";

newStudent.lastName = "Brown";
```

store **"John"** in the member **firstName** and **"Brown"** in the member **lastName** of **newStudent.**

After the preceding three assignment statements execute, **newStudent** is as shown in Figure.



The statement:

```
Cin>> newStudent.firstName;
```

reads the next string from the standard input device and stores it in:

newStudent.firstName

The statement:

```
Cin>> newStudent.testScore ;
Cin>>newStudent.programmingScore;
```

reads two integer values from the keyboard and stores them in **newStudent.testScore** and **newStudent.programmingScore**, respectively.

Suppose that **score** is a variable of type **int**. The statement:

```
score = (newStudent.testScore + newStudent.programmingScore)/ 2;
```

assigns the average of **newStudent.testScore** and **newStudent.programmingScore** to score.

The following statement determines the course grade and stores it in **newStudent.courseGrade**:

```
if(score >= 90)

newStudent.courseGrade = 'A';

else if(score >= 80)

newStudent.courseGrade = 'B';

else if(score >= 70)

newStudent.courseGrade = 'C';

else if(score >= 60)

newStudent.courseGrade = 'D';

else

newStudent.courseGrade = 'F';
```

Example 1:-

Write a program to input data into and then print data from the members of the structure.

```
#include <stdio.h>

#include <stdlib.h>
#include <string.h>
struct address

{

char city [15];
 int pcode;

};



int main()

{

struct address taq;
cout<<”Enter City";
cin>>taq.city;
taq.pcode=123;

cout<<"Output from structure is ";
cout<<"\n;
cout<<"City is: ”<<taq.city;
   cout<<”/n");

cout<<”Postal Code is "<<taq.pcode;

}
```

Example 2:-

Write a program to copy one structure variable to another variable and then to print it on the screen.

```
#include <iostream>
```

Lab # 04

```
#include <stdlib.h>
#include <string.h>

struct abc

{

char name [15]; int age;

};


int main()

{

struct  abc s1,s2,t; int i;

cout <<"Enter Name: ";
cin>>"s1.name;

cout<<"Enter Age: ";
 cin>>s1.age;

s2.age=s1.age;

for(i=0;s1.name[i]!='\0';i++)
{
s2.name[i]=s1.name[i];
s2.name[i]='\0';
}

Cout<<"Name in s2 is:\n"<<s2.name;
Cout<<"Age in s2 is:  "<<s2.age;


}
```

Initialization of Structure Variables:-

The values into a structure variable can be assigned when it is declared. It is called initialization of the structure variable.

```
struct address

        {
```

```
        char city[15];
        int pcode;
    };
address taq = {"Lahore" , 54500};
```

Structs within a struct:-

You have seen how the **struct** and array data structures can be combined to organize information. You also saw examples wherein a member of a **struct** is an array, and the array type is a **struct**. Now we will learn about situations for which it is beneficial to organize data in a **struct** by using another **struct**.
Let us consider the following employee record:

```
struct employeeType
{
string firstname;
stringmiddlename;
string lastname;
string empID;
string address1;
string address2;
string city;
string state;
string zip;
int hiremonth;
int hireday;
int hireyear;
```

```
int quitmonth;

int quitday;

int quityear;

string phone;

string cellphone;

string fax;

string pager;

string email;

string deptID;

double salary;};
```

As you can see, a lot of information is packed into one **struct**. This **struct** has 22 members. Some members of this **struct** will be accessed more frequently than others, and some members are more closely related than others. Moreover, some members will have the same underlying structure. For example, the hire date and the quit date are of the date type **int**. Let us reorganize this **struct** as follows:

```
struct nameType

{

String first;

string middle;

string last;


};

struct addressType

{
string address1;
```

```
string address2;

string city;

string state;

string zip;

};
struct dateType

{
int month;

int day;

int year; };


struct contactType
{

string phone;

string cellphone;

string fax;

string pager;

string email;

};
```

We have separated the employee's name, address, and contact type into subcategories. Furthermore, we have defined a **`struct dateType`**. Let us rebuild the employee's record as follows:

```
struct employeeType

{
nameType name;

string empID;
```

```
addressType address;

dateType hireDate;

dateType quitDate;

contactType contact;

string deptID;

double salary;

};
```

The information in this **struct** is easier to manage than the previous one. Some employee's of this **struct** can be reused or example, suppose that you want to define a customer's record. Every customer has a first name, last name, and middle name, as well as an address and a way to be contacted. You can, therefore, quickly put together a customer's record by using the **structs** **nameType, addressType, contactType,** and the members specific to the customer.

Next, let us declare a variable of type **employeeType** and discuss how to access its members.

Consider the following statement:

This statement declares **newEmployee** to be a **struct** variable of type **employeeType**

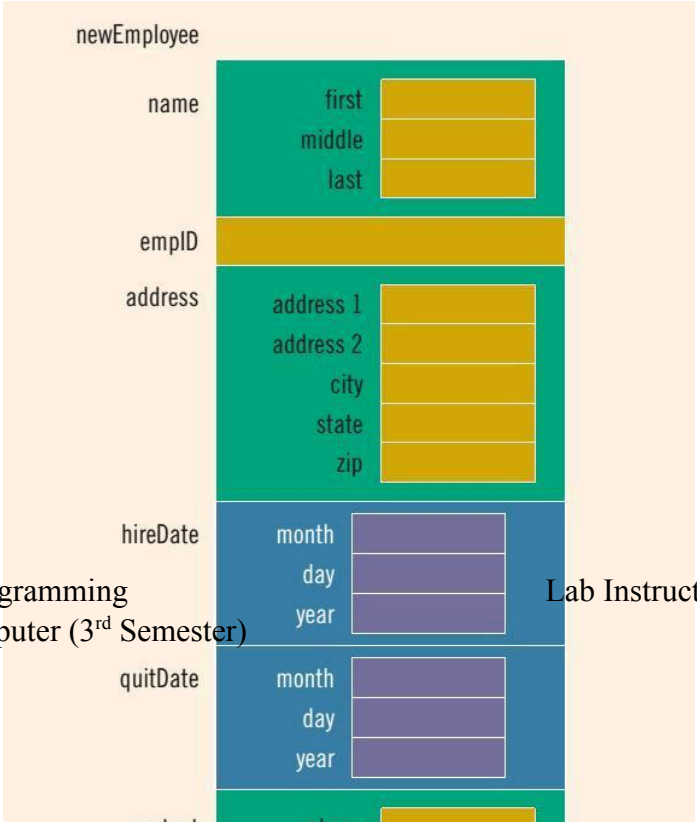


Figure 6: struct variable newEmployee

The statement:

```
newEmployee.salary = 45678.00;
```

sets the salary of newEmployee to 45678.00. The statements:

```
newEmployee.name.first = "Mary";

newEmployee.name.middle = "Beth";

newEmployee.name.last = "Simmons";
```

set the **first**, **middle**, and **last** name of **newEmployee** to **"Mary"**, **"Beth"**, and **"Simmons"**, respectively. Note that **newEmployee** has a member called name. We access this member via **newEmployee.name.** Note also that **newEmployee.name** is a struct and has three members. We apply the member access criteria to access the member first of the struct **newEmployee.name**. So, **newEmployee.name.first** is the member where we store thefirst name.

The statement:

```
Cin>> newEmployee.name.first;
```

reads and stores a string into **newEmployee.name.first**. The statement:

```
newEmployee.salary = newEmployee.salary * 1.05;
```

updates the salary of **newEmployee.**

The following statement declares employees to be an array of 100 components, wherein each component is of type **employeeType**:

```
employeeType employees[100];
```

The for loop:

```
for(int j = 0; j < 100; j++)
{
cin>>employees[j].name.first>>employees[j].name.middle>>e

mployees[j].name.last;

}
```

reads and stores the names of 100 employees in the array employees. Because employees is an array, to access a component, we use the index. For example, **employees[50]** is the 51st component of the array employees(recall that an array index starts with 0). Because **employees[50]** is a struct, we apply the member access criteria to select a particular member.

Lab Tasks

1. Write a complete program in C++ to run above example i.e. record of a new hiring employee.
   a. Set all record of new employee by passing argument.
   b. Display all record of new employee.
2. Write a program in C++ to define a Structure of point (x, y) and calculate the distance between two points. Use Distance calculation formula. Square Root((x2-x1)^2+(y2-y1)^2))
   a) Define a point p1 (10, 4) and p2 (12, 6) and find the distance using given information.
3. Write a program in C++ to find the height difference of two persons in feet and inches. (Hint: 1ft=12 inches)
4. Write a program in C++ to add two complex numbers.
   (**NOTE: Use functions**)

Conclusion:

HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 04

_____
_____
_____
_____
_____
_____

_____
Lab Instructor
Kaynat Rana